

Imports and Perceptron class

```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    """
    Rosenblatt perceptron with bias and hard limiter activation.
     $y = 1$  if  $w^T x + b \geq 0$  else 0
    """

    def __init__(self, input_dim, lr=0.1, max_epochs=50, random_state=None):
        self.input_dim = input_dim
        self.lr = lr
        self.max_epochs = max_epochs
        self.rng = np.random.default_rng(random_state)
        # Small random init
        self.w = self.rng.uniform(-0.5, 0.5, size=(input_dim,))
        self.b = self.rng.uniform(-0.5, 0.5)
        self.epoch_errors = []

    def _step(self, net):
        return (net >= 0).astype(int)

    def net_input(self, X):
        return X @ self.w + self.b

    def predict(self, X):
        return self._step(self.net_input(X))

    def fit(self, X, y, verbose=False):
        """
        Online Rosenblatt update until:
        - zero errors in an epoch, or
        - max_epochs reached
        """
        X = np.asarray(X, float)
        y = np.asarray(y, int)
        n_samples = X.shape[0]
        self.epoch_errors = []

        for epoch in range(self.max_epochs):
            errors = 0
            for i in range(n_samples):
                xi = X[i]
                target = y[i]
                net = float(np.dot(self.w, xi) + self.b)
                pred = 1 if net >= 0 else 0
                e = target - pred
                if e != 0:
                    # Update rule
                    self.w += self.lr * e * xi
```

```

        self.b += self.lr * e
        errors += 1
    self.epoch_errors.append(errors)
    if verbose:
        print(f"Epoch {epoch+1}: errors = {errors}")
    if errors == 0:
        break

    return epoch + 1 # number of epochs actually used

```

Datasets for AND, OR, NOT, XOR (4 samples each)

```

def get_gate_data(gate_name):
    """
    Return X, y for logic gate.
    We keep 4 samples for all:
    - AND, OR, XOR: standard 2D inputs
    - NOT: embed 1D NOT into 2D with x2 = 0, duplicated to get 4 points
    """
    gate_name = gate_name.upper()

    if gate_name in ["AND", "OR", "XOR"]:
        # Standard 2-input 4-sample dataset
        X = np.array([[0, 0],
                      [0, 1],
                      [1, 0],
                      [1, 1]], dtype=float)
        if gate_name == "AND":
            y = np.array([0, 0, 0, 1], dtype=int)
        elif gate_name == "OR":
            y = np.array([0, 1, 1, 1], dtype=int)
        elif gate_name == "XOR":
            y = np.array([0, 1, 1, 0], dtype=int)

    elif gate_name == "NOT":
        # Single variable NOT: 0 -> 1, 1 -> 0
        # Embed as 2D: (0,0)->1, (1,0)->0, and duplicate to have 4 samples
        X = np.array([[0, 0],
                      [1, 0],
                      [0, 0],
                      [1, 0]], dtype=float)
        y = np.array([1, 0, 1, 0], dtype=int)

    else:
        raise ValueError(f"Unsupported gate: {gate_name}")

    return X, y

```

Helper to train and print results for one gate

```

def train_and_show_gate(gate_name, lr=0.1, max_epochs=50, random_state=0, show_plot=True):
    X, y = get_gate_data(gate_name)
    input_dim = X.shape[1]

    p = Perceptron(input_dim=input_dim, lr=lr, max_epochs=max_epochs, random_state=random_state)
    epochs_used = p.fit(X, y, verbose=False)
    y_pred = p.predict(X)
    acc = (y_pred == y).mean()

    print(f"=== {gate_name} gate ===")
    print("Inputs X:")
    print(X)
    print("Targets y:", y)
    print("Predictions:", y_pred)
    print(f"Final weights w: {p.w}")
    print(f"Final bias b: {p.b}")
    print(f"Epochs until stop: {epochs_used}")
    print(f"Training accuracy: {acc * 100:.1f}%")
    print("Errors per epoch:", p.epoch_errors)
    print()

    # Optional: plot decision boundary for 2D gates
    if show_plot and X.shape[1] == 2:
        plot_decision_boundary(p, X, y, title=f"{gate_name} gate decision boundary")

    return p, epochs_used, acc

def plot_decision_boundary(perceptron, X, y, title=""):
    X = np.asarray(X, float)
    y = np.asarray(y, int)
    w = perceptron.w
    b = perceptron.b

    plt.figure(figsize=(4, 4))

    # Plot points (class 0 = red 'o', class 1 = blue 'x')
    for xi, yi in zip(X, y):
        if yi == 0:
            plt.scatter(xi[0], xi[1], color="red", marker="o", label="0" if "0" not in plt.gca().get_legend_handles_labels()[0])
        else:
            plt.scatter(xi[0], xi[1], color="blue", marker="x", label="1" if "1" not in plt.gca().get_legend_handles_labels()[0])

    # Line: w1 x1 + w2 x2 + b = 0 => x2 = -(w1 x1 + b) / w2
    x_vals = np.linspace(-0.5, 1.5, 200)
    if abs(w[1]) > 1e-8:
        y_vals = -(w[0] * x_vals + b) / w[1]
        plt.plot(x_vals, y_vals, "k-", label="decision boundary")
    else:
        # Vertical line x1 = -b / w1
        x_line = -b / w[0]
        plt.axvline(x_line, color="k", label="decision boundary")

    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.xlabel(r"$x_1$")

```

```
plt.ylabel(r"$x_2$")
plt.title(title)
plt.legend()
plt.grid(True)
plt.show()
```

Run in required order: AND → OR → NOT → XOR

```
results = {}

# 1. AND
p_and, e_and, a_and = train_and_show_gate("AND")
results["AND"] = {"perceptron": p_and, "epochs": e_and, "acc": a_and}

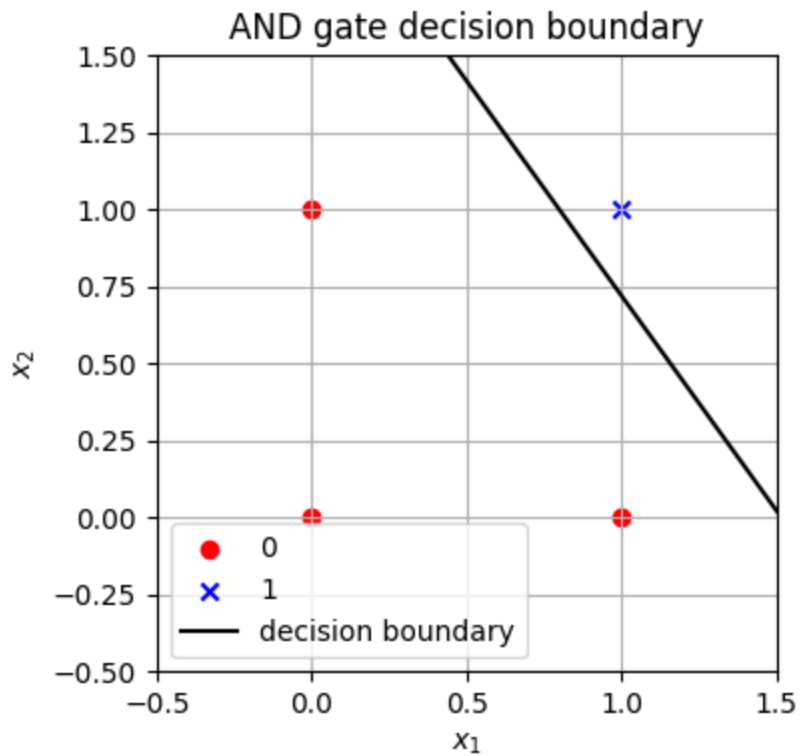
# 2. OR
p_or, e_or, a_or = train_and_show_gate("OR")
results["OR"] = {"perceptron": p_or, "epochs": e_or, "acc": a_or}

# 3. NOT (embedded in 2D)
p_not, e_not, a_not = train_and_show_gate("NOT")
results["NOT"] = {"perceptron": p_not, "epochs": e_not, "acc": a_not}

# 4. XOR (expected to fail: not linearly separable)
p_xor, e_xor, a_xor = train_and_show_gate("XOR")
results["XOR"] = {"perceptron": p_xor, "epochs": e_xor, "acc": a_xor}

print("Summary (epochs, accuracy):")
for g, info in results.items():
    print(f"{g:3s}: epochs={info['epochs']:2d}, acc={info['acc']*100:4.1f}%")

=== AND gate ===
Inputs X:
[[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]]
Targets y: [0 0 0 1]
Predictions: [0 0 0 1]
Final weights w: [0.23696169 0.16978671]
Final bias b: -0.35902647606380533
Epochs until stop: 6
Training accuracy: 100.0%
Errors per epoch: [1, 1, 2, 2, 1, 0]
```



=== OR gate ===

Inputs X:

[[0. 0.]

[0. 1.]

[1. 0.]

[1. 1.]]

Targets y: [0 1 1 1]

Predictions: [0 1 1 1]

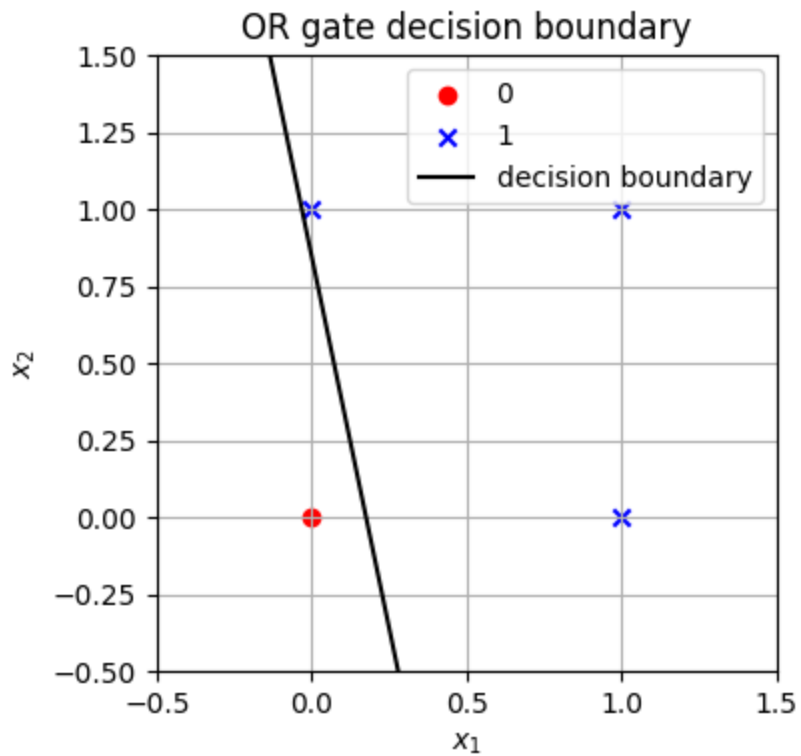
Final weights w: [0.33696169 0.06978671]

Final bias b: -0.059026476063805344

Epochs until stop: 3

Training accuracy: 100.0%

Errors per epoch: [3, 1, 0]



=== NOT gate ===

Inputs X:

[[0. 0.]

[1. 0.]

[0. 0.]

[1. 0.]]

Targets y: [1 0 1 0]

Predictions: [1 0 1 0]

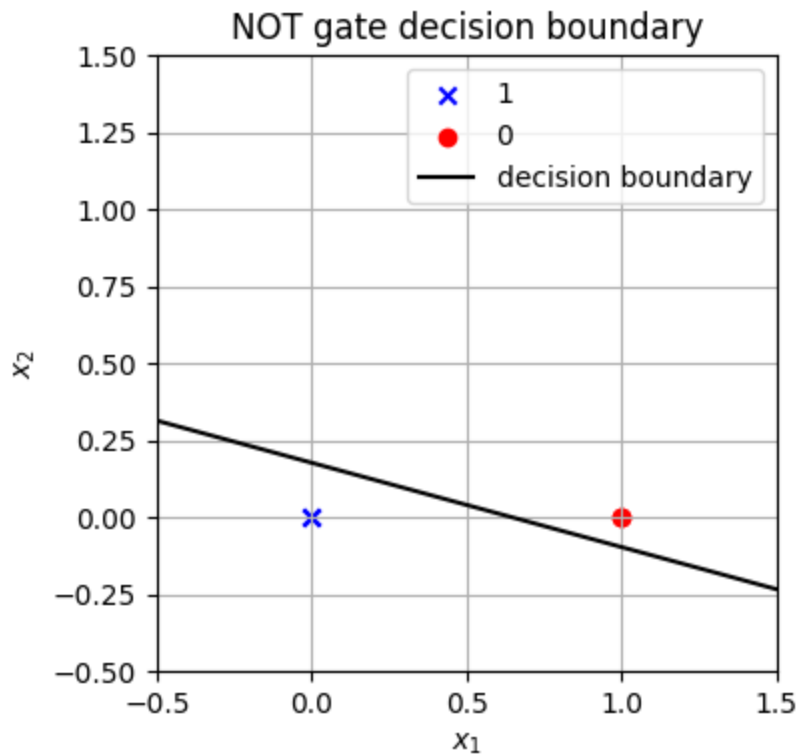
Final weights w: [-0.06303831 -0.23021329]

Final bias b: 0.04097352393619466

Epochs until stop: 5

Training accuracy: 100.0%

Errors per epoch: [2, 3, 3, 1, 0]



=== XOR gate ===

Inputs X:

[[0., 0.]]

[0. 1.]

[1. 0.]

```
[1. 1.]
```

Targets y: [0 1 1 0]

```
Predictions: [1 0 0 0]
```

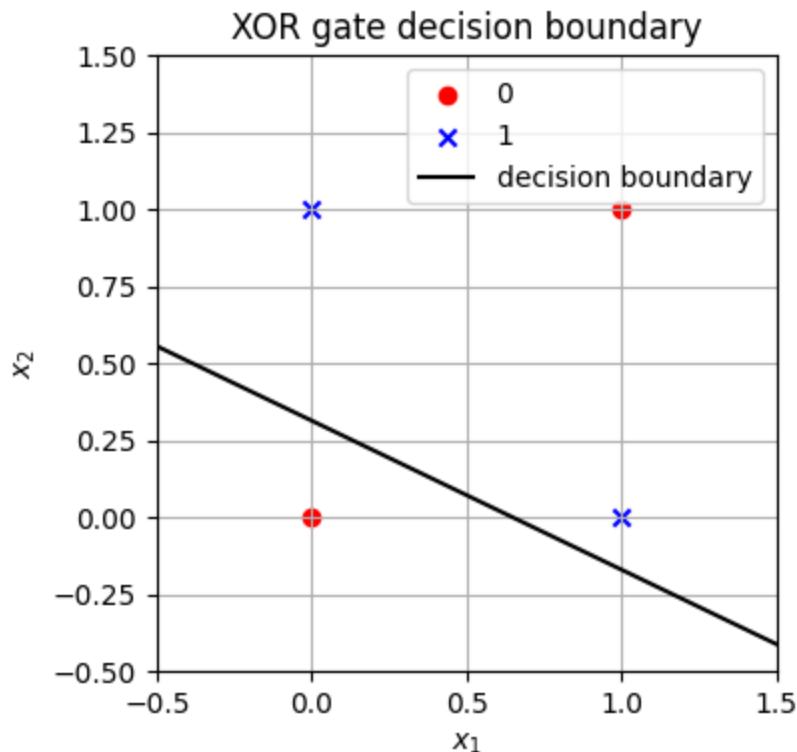
Final weights w : $[-0.06303831 \ -0.13021329]$

Final bias b: 0.04097352393619466

Epochs until stop: 50

Training accuracy: 25.0%

```
Errors per epoch: [2, 2, 3, 2, 3, 2, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
```



Summary (epochs, accuracy):

AND: epochs= 6, acc=100.0%

OR : epochs= 3, acc=100.0%

NOT: epochs= 5, acc=100.0%

XOR: epochs=50, acc=25.0%

Theory: Why AND, OR, NOT Work with a Single Perceptron and XOR Fails

1. Perceptron = Linear Classifier

A perceptron with weights $\mathbf{w} = [w_1, w_2]^T$ and bias b computes:

[y =

$$\begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

]

Geometrically, the equation $[w_1x_1 + w_2x_2 + b = 0]$ is a straight line in the (x_1, x_2) plane. The perceptron assigns class 1 to all points on one side of this line and class 0 to all points on the other side. So a single perceptron can only solve problems where the two classes can be separated by a single straight line – this property is called **linear separability**.

2. Why AND Is Linearly Separable

AND gate outputs 1 only at $(1, 1)$:

- Class 0: $(0, 0), (0, 1), (1, 0)$
- Class 1: $(1, 1)$

We can find a line that puts $(1, 1)$ alone on the 1-side and all others on the 0-side. For example:

$$[w = [1, 1], \text{quad } b = -1.5]$$

gives:

- $(0, 0): 1 \cdot 0 + 1 \cdot 0 - 1.5 = -1.5 < 0 \Rightarrow y = 0$
- $(0, 1): 1 \cdot 0 + 1 \cdot 1 - 1.5 = -0.5 < 0 \Rightarrow y = 0$
- $(1, 0): 1 \cdot 1 + 1 \cdot 0 - 1.5 = -0.5 < 0 \Rightarrow y = 0$
- $(1, 1): 1 \cdot 1 + 1 \cdot 1 - 1.5 = 0.5 \geq 0 \Rightarrow y = 1$

So a single straight line works $\Rightarrow$ AND is **linearly separable**.

3. Why OR Is Linearly Separable

OR gate outputs 1 everywhere except $(0, 0)$:

- Class 0: $(0, 0)$
- Class 1: $(0, 1), (1, 0), (1, 1)$

One separating line is:

$$[w = [1, 1], \text{quad } b = -0.5]$$

- $(0, 0): -0.5 < 0 \Rightarrow y = 0$
- $(0, 1): 0.5 \geq 0 \Rightarrow y = 1$
- $(1, 0): 0.5 \geq 0 \Rightarrow y = 1$
- $(1, 1): 1.5 \geq 0 \Rightarrow y = 1$

Again, a single line separates the classes $\Rightarrow$ OR is **linearly separable**.

4. Why NOT Is Linearly Separable

NOT is a 1D gate. Embedding in 2D as we did:

- Inputs: $(0, 0) \mapsto 1, (1, 0) \mapsto 0$ (duplicated to get 4 points)
- So on the x -axis, we just need to put 0 on one side of a threshold and 1 on the other.

For example, in 1D:

$$[w = -1, \text{quad } b = 0.5]$$

- $x = 0: -1 \cdot 0 + 0.5 = 0.5 \geq 0 \Rightarrow y = 1$
- $x = 1: -1 \cdot 1 + 0.5 = -0.5 < 0 \Rightarrow y = 0$

One threshold (a point in 1D, or a vertical line in 2D) is enough $\Rightarrow$ NOT is **linearly separable**.

5. Why XOR Is *Not* Linearly Separable

XOR labels:

- Class 0: $(0, 0)$, $(1, 1)$
- Class 1: $(0, 1)$, $(1, 0)$

Geometrically:

- The positive points $(0, 1)$ and $(1, 0)$ are on opposite corners.
- The negative points $(0, 0)$ and $(1, 1)$ are on the other two opposite corners.

Any straight line in 2D divides the plane into two half-planes. There is no way to draw a single straight line so that **both** positives are on one side and **both** negatives are on the other, because:

- If the line separates $(0, 0)$ from $(0, 1)$, it must cross between them vertically.
- If it also separates $(0, 0)$ from $(1, 0)$, it must cross between them horizontally.
- Trying to satisfy both constraints forces the line layout to misclassify at least one of $(1, 1)$ or a positive point.

More formal statement: the convex hull of positive points intersects the convex hull of negative points, so no linear separator exists.

Therefore, **no choice** of (w_1, w_2, b) can make a single perceptron implement XOR exactly. Training will not converge to 100% accuracy; the algorithm will keep making errors.

6. Connection to Deep Learning

- A single perceptron is a **linear model** (like logistic regression).
- AND, OR, NOT are simple examples where a linear decision boundary is enough.
- XOR is the classic example that shows you need **more than one linear boundary** – i.e., a **hidden layer** combining several perceptrons.
- In a multilayer perceptron (an MLP), the first layer can carve the space into multiple regions (nonlinear representation), and the final layer can then linearly separate those regions. This is exactly the kind of “learn a representation, then use a linear classifier” idea emphasized in modern deep learning.

In summary:

- AND, OR, NOT can be separated by a single straight line (or threshold) $\Rightarrow$ a single perceptron learns them.
- XOR cannot be separated by any single straight line $\Rightarrow$ a single perceptron will always fail on XOR; you need at least a 2-layer network. ""